

	NOTA TECNICA TECHNICAL REPORT		Núm. No. NT-T-ADP-04002	
			Pág. 1 de 33 Page 1 of 33	Edic. Issue A
	Departamento Department	Métodos	Avión Aircraft General	
Título/Title Generación de librerías PBARL para MSC.NASTRAN				
Palabras clave/Key words FINITE ELEMENT METHOD, LIBRARIES, NASTRAN			Clasificación acceso Access class P1	
Resumen/Summary El objetivo de esta nota técnica es servir de guía en el uso y desarrollo de librerías de secciones de vigas PBARL. El propósito es crear, e incluir en MSC.NASTRAN, secciones alternativas a las disponibles en la librería estándar de MSC.NASTRAN Esta aplicación permite una mejora en la creación y modificación de las tarjetas correspondientes a propiedades de vigas con secciones muy utilizadas, por ejemplo, en la modelización de cuadernas. Asimismo, simplifica de modo muy importante la tarea de chequeo de dichas tarjetas frente al PBAR convencional. Se incluye una descripción tanto del proceso generador de las librerías como del uso de las mismas. La librería generada tiene aplicación para la versión 70.5 de MSC.NASTRAN Copia de archivo sin firmas. Archive copy without signatures. Para obtener las páginas firmadas, If Page with signatures are required, por favor contacte con: please contact: EADS CASA Dirección de Proyectos y Sistemas Servicios Técnicos de Documentación PROPIEDAD DE EADS CASA. Este documento no puede ser utilizado ni reproducido total o parcialmente sin la previa autorización escrita de la Dirección de Proyectos y Sistemas de EADS CASA. EADS CASA PROPERTY. This document shall neither be used nor completely or partially reproduced without previous written authorisation by EADS CASA Engineering and System Direction.				
Distribución/Distribution	Realización/Execution		Fecha/Date 30 Marzo 2004	
Archivo de Cálculo	Realizado Prepared		Firma Signature	
J.M. Ruis Barragán (*)	Julián López Garrido			
Jefes Departamento Cálculo (*)				
Jefes Sección Cálculo (*)				
J. L. Melchor				
	Comprobado Checked	Eduardo OSLÉ DORREMOCHEA		
(*) Sólo Portada	Aprobado Approved	Agustín GONZÁLEZ DÍAZ		

REGISTRO DE REVISIONES/REVISIONS RECORD

[illegible]

INDICE

1 Introducción

1.1 Descripción general	1.1.1
1.2 Referencias.....	1.2.1

2 Utilización de la librería

2.1 Utilización.....	2.1.1
2.2 Limitaciones.....	2.2.1

3 Generación de la librería

3.1 Ubicación de la plantilla	3.1.1
3.2 Creación de archivos auxiliares	3.2.1
3.3 Edición de registro	3.3.1
3.4 Descripción de componentes	3.4.1
3.5 Edición de propiedades.....	3.5.1
3.6 Compilación.....	3.6.1

4 Instalación

4.1 Ubicación e instalación de la librería.....	4.1.1
---	-------

5 Validación de resultados

5.1 Ejemplo 1: equivalencia sección C Variable con CHAN	5.1.1
5.2 Ejemplo 2: sección C Variable calculada con AIDA	5.2.1
5.3 Ejemplo 3: sección G Variable calculada con AIDA	5.3.1

1 Introducción

El objetivo de esta nota técnica es servir de guía en el uso y desarrollo de librerías de secciones de vigas PBARL. El propósito es crear, e incluir en MSC.NASTRAN, secciones alternativas a las disponibles en la librería estándar de MSC.

Esta aplicación permite una mejora en la creación y modificación de las tarjetas correspondientes a propiedades de vigas con secciones muy utilizadas, por ejemplo, en la modelización de cuadernas.

Se incluye una descripción tanto del proceso generador de las librerías como del uso de las mismas.

Se ha creado una librería de vigas que contiene dos tipos de secciones muy utilizadas en las cuadernas de fuselajes: C con cordones reforzados y G (cuaderna típica de chapa).

La librería generada tiene aplicación para la versión 70.5 de MSC.NASTRAN y será fácilmente extensible a futuras versiones.

Este documento pretende abordar el objetivo principal desde dos puntos de vista, el primero como usuario, y el segundo como generador de nuevas secciones. Para este segundo tipo de usuario están destinadas la mayor parte de páginas, mientras que el potencial usuario deberá centrarse en el segundo apartado.

En el apartado 5 se han incluido comprobaciones de varias secciones, comparando los resultados con los obtenidos con el programa AIDA.

1.1 Descripción general

Una librería PBARL consiste en, una serie de rutinas que calculan las propiedades geométricas de una sección de dimensiones variables.

El objetivo de estas librerías es ahorrar la tarea de calcular dichas propiedades, de forma previa al lanzamiento de nuestro modelo. Las ventajas son muchas, desde el control y verificación de dimensiones hasta la variación de la sección (de otro modo, si modificamos cualquier dimensión, todas las propiedades quedan afectadas), pasando por la optimización.

El desarrollo sigue la filosofía de trabajo preestablecida por MSC.NASTRAN. Las librerías estarán estructuradas como la librería por defecto de MSC.NASTRAN, LOCBMLS0. Los archivos que compondrán la librería, cuando esta se compile, dependerán del número de secciones que contenga la librería, por lo que el tamaño de esta dependerá de su extensión.

A lo largo del documento se hace referencia a un ejemplo de librería, la cual es la librería utilizada en el desarrollo. Dicha librería contiene dos secciones, para las cuales se describirá el proceso paso a paso.

1.2 Referencias

- Ref. 1 MSC.NASTRAN RELEASE GUIDE - V69 APPENDIX C
- Ref. 2 MSC.NASTRAN Version 70.7 Configuration and Operations Guide Chapter 7.2
- Ref. 3 MSC.NASTRAN Version 70.5 Quick Reference Guide
- Ref. 4 MSC.NASTRAN Version 70.5, sistema operativo SUN SPARC SunOS 5.8
- Ref. 5 AIDA Version 2.2, sistema operativo Windows NT
- Ref. 6 Save set NT-T-ADP-04002

2 Utilización de la librería

Como anteriormente se ha mencionado, se utilizará una librería ejemplo para la ilustración del manejo de la aplicación. Dicha librería se llamará LOCBMLS1, contendrá las propiedades CV y GV correspondientes a las secciones "C variable" y "G variable".

2.1 Utilización

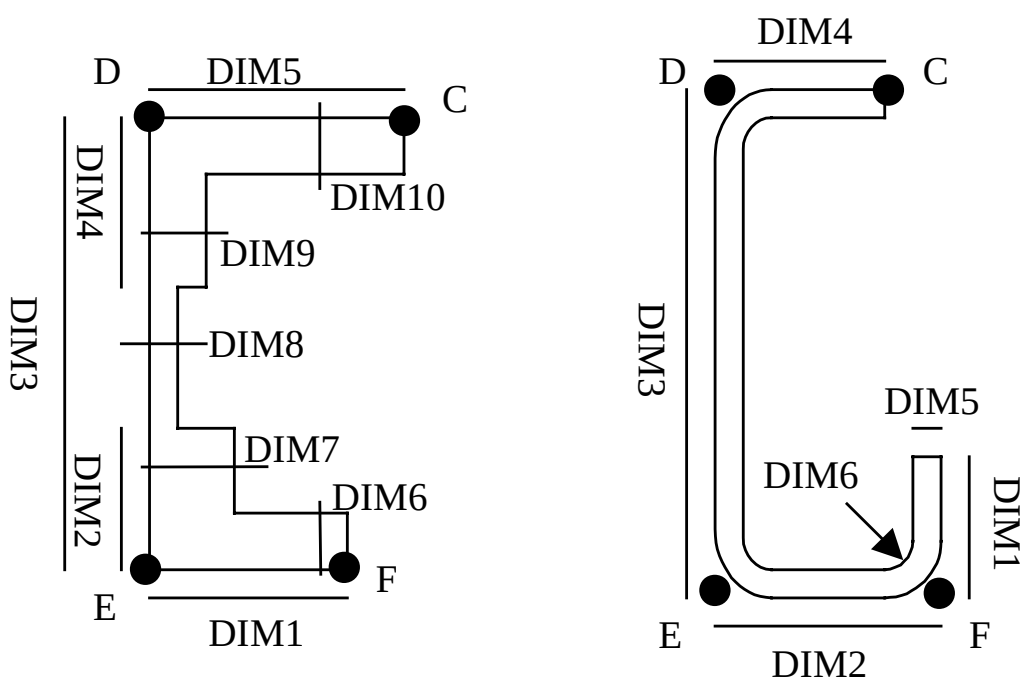


Figura 2.1.1: Geometría de secciones CV y GV.

La utilización de una librería nueva será similar a la de cualquier librería MSC.NASTRAN, con un par de salvedades que apuntaremos tras la redacción de las tarjetas. Actualmente se ha creado la librería LOCBMLS1, la cual contiene dos secciones distintas: Sección CV (C variable) y Sección GV (G variable).

Para poder utilizar las propiedades de una sección debemos generar las tarjetas PBARL correspondientes. Una tarjeta es la manera que tiene MSC.NASTRAN para asimilar y expulsar datos. Está dividida en una serie de líneas (normalmente consecutivas) de la base de datos, que puede asimilarse como un archivo .bdf y cada línea, a su vez está dividida en campos. La longitud, en caracteres, depende del formato utilizado, pero, independientemente del formato, cada línea contiene 6 o 10 campos (doble o simple precisión).

La estructura de la PBARL recomendada es la siguiente:

```

$234567$234567$234567$234567$234567$234567$234567$234567$234567$234567$
PBARL  PID      MID      GROUP  TYPE      (BLANK) (BLANK) (BLANK) (BLANK) (BLANK)
(BLANK) DIM1    DIM2    DIM3    DIM4    DIM5    DIM6    DIM7    DIM8    (BLANK)
(BLANK) DIM9    -etc.-  Switch  NSM
  
```

La primera línea se utiliza para tener una referencia de la longitud de cada campo, no influye en la base de datos porque es una línea comentada.

La siguiente línea indica el tipo de tarjeta, en este caso:

- PBARL es la definición de propiedades de barra por librería.
- PID es el número de identificación de la propiedad.
- MID es el número de identificación de material.
- GROUP es el grupo de la sección (en caso de utilizar la librería por defecto de MSC.NASTRAN, este campo sería un campo BLANK).
- TYPE es el tipo de sección.
- BLANK son campos vacíos equivalentes a ocho espacios.
- DIM1-DIMi son las dimensiones de la sección.
- NSM es la masa no estructural por unidad de longitud.
- **Switch** es un campo nuevo que se ha introducido en la librería EADS-CASA. Consiste en una opción que cambia el proceso de cálculo de la sección completa a limitarse al área (A) y la primera inercia (I_{ZZBAR}) de la sección, que es el modo de trabajar usual para los elementos CBAR de las cuadernas de Fuselaje. Recordemos que I_{ZZBAR} es la inercia que soporta los momentos flectores contenidos en el plano 1 del elemento CBAR, esto es, el plano definido por el eje x del CBAR y el vector orientación.
- El campo Switch (Real) ha de ser rellenado con un valor 0. (real) para la sección completa y 1. (real). Si sólo queremos A e I_{ZZBAR} .

Dentro de cada librería, y para el uso de una sección concreta, deberemos de conocer una serie de propiedades para poder escribir su tarjeta:

- Nominación del tipo de sección (c variable $\rightarrow$ CV, g variable $\rightarrow$ GV)
- Número de campos de la tarjeta (CV $\rightarrow$ 10 geométricos + Switch + NSM = 12, GV $\rightarrow$ 6 geométricos + Switch + NSM = 8) de los cuales el último, normalmente, podrán dejarse en blanco
- *path* donde se ubica la librería.

Una vez conocidos estos datos, se deben seguir los siguientes pasos:

1. Se introduce la tarjeta correspondiente

Los campos siguientes, para uso de la librería EADS-CASA, se recomienda que se rellenen del siguiente modo:

-GROUP = **LOCBMLS1**

-TYPE = **CV** ó **GV**, dependiendo de la sección a utilizar

-DIMi de acuerdo con la figura 2.1.1

-Switch = **0.** para cálculo completo, **1.** para cálculo simplificado (Real)

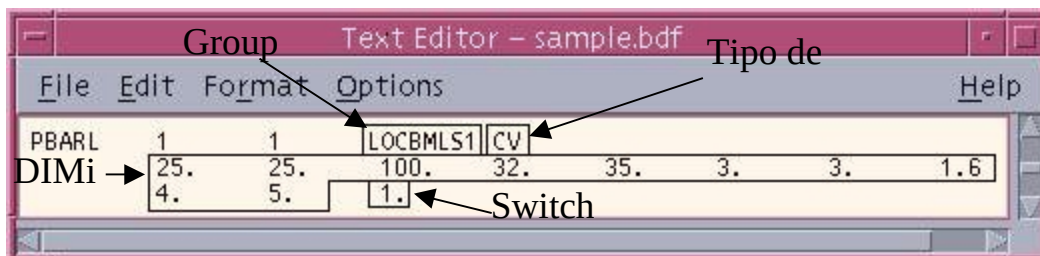


Figura 2.1.2: Tarjeta sección CV.

2. Se introduce en la sección FMS (File Management Section, ref 3) del archivo .bdf del modelo

CONNECT BEAMEVAL *group evaluator*

En el caso de librería EADS-CASA *group* = **LOCBMLS1** y *evaluator* = **LOCBMLS**.

CONNECT BEAMEVAL LOCBMLS1 LOCBMLS

3. Se ejecuta MSC.NASTRAN con un comando ampliado al utilizado habitualmente

nastran *model.bdf gmconn=archive*

En el caso de librería EADS-CASA *archive* = **/homesun/locus/cal/msc/old/msc/samp_eval**

NASTRAN MODEL.BDF GMCONN=/HOMESUN/LOCUS/CAL/MS/OLD/MS/SAMP_EVAL

Todos las palabras en cursiva deberán ser sustituidas por los elementos a los que hacen referencia. *Group* se refiere al nombre del grupo al cual pertenecen las secciones de la librería (LOCBMLS1), y deberá ser el mismo en la instrucción **CONNECT** y en el tercer campo de la tarjeta. *Evaluator* es el nombre de la librería (LOCBMLS en el ejemplo), este nombre quedará definido por el archivo *archive*, que será un documento de texto (nombre libre) ubicado donde vayamos a ejecutar MSC.NASTRAN, en el cual deberemos incluir una línea con el siguiente formato:

Evaluator,-,path

Path es el dato que habíamos solicitado anteriormente, y únicamente es la dirección donde se ubica la librería. Un ejemplo de path sería **/homesun/locus/cal/msc/old/msc/msc705/bmsrv/beamserve** para una librería beamserv. Model es el nombre de nuestro modelo, normalmente un archivo .bdf pero que en este caso necesita una herramienta externa.

Se adjunta Bulk Data utilizado en el test de la librería a modo de ejemplo:

```
CONNECT BEAMEVAL LOCBMLS1 LOCBMLS
SOL 101
CEND
SEALL = ALL
SUPER = ALL
TITLE = MSC.Nastran job created on 16-Jan-04 at 14:01:57
ECHO = punch
SUBCASE 1
SUBTITLE=Default
    SPC = 2
    LOAD = 2
```



```

DISPLACEMENT(SORT1,REAL)=ALL
SPCFORCES(SORT1,REAL)=ALL
STRESS(SORT1,REAL,VONMISES,BILIN)=ALL
BEGIN BULK
PARAM      POST      0
PARAM      AUTOSPC YES
PARAM      PRTMAXIM YES
PBARL*  1              1              LOCBMLS1      GV
*
*      .5              1.              2.              .7
*      .1              .1              .0
CBAR      1      1      1      2      0.      1.      0.
CBAR      2      1      2      3      0.      1.      0.
CBAR      3      1      3      4      0.      1.      0.
CBAR      4      1      4      5      0.      1.      0.
MAT1      1      70000.      .3      2.
GRID      1      0.      0.      0.
GRID      2      .25      0.      0.
GRID      3      .5      0.      0.
GRID      4      .75      0.      0.
GRID      5      1.      0.      0.
SPCADD      2      1
LOAD      2      1.      1.      1
SPC1      1      123456      1
FORCE      1      5      0      1.-3      1.      0.      0.
ENDDATA d885da65

```

2.2 Limitaciones

Esta herramienta pertenece al entorno MSC.NASTRAN, por lo que ningún otro programa reconocerá ninguno de los campos relacionados con la librería. Por tanto, un modelo que incluya alguna sección no contenida en la librería de MSC.NASTRAN no podrá importarse en PATRAN directamente, ni en ningún otro programa similar. La solución a este tipo de problemas es correr el modelo bajo MSC.NASTRAN y sustituir las tarjetas PBARL por las PBAR equivalentes que devuelve MSC.NASTRAN en el archivo .pch, suprimiendo la línea de conexión a la librería (primera línea del modelo).

En ningún caso se podrá lanzar el modelo si no se han cumplido todos los requisitos del apartado anterior, por lo que se debe tener especial cuidado con omitir el archivo o su llamada en la instrucción de lanzamiento.

No deberán excederse ninguna de las limitaciones geométricas de las secciones, como por ejemplo poner dimensiones negativas o iguales a cero, un radio exterior menor a uno interior, o relacionadas con el ejemplo, en la sección CV, la suma de las dimensiones 2 y 4 mayor que la dimensión 3. Este tipo de descuidos impedirán que continúe el proceso de cálculo y vendrán reflejados como mensaje de error en el archivo .f06 correspondiente al modelo.

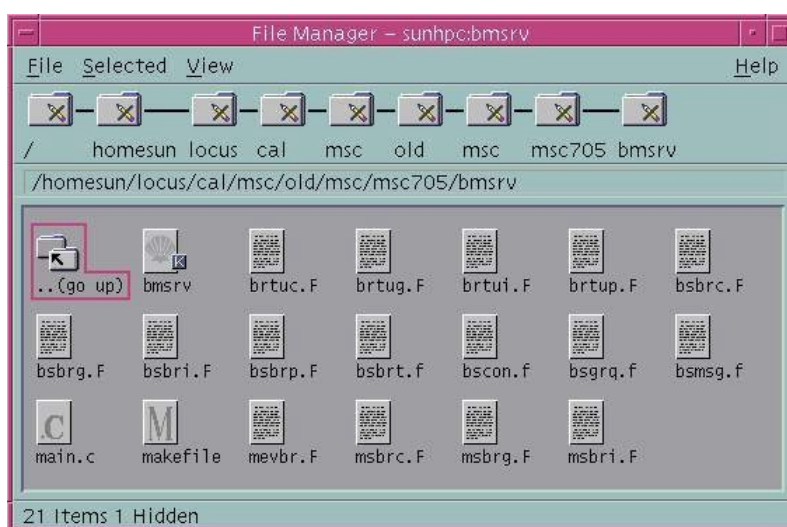
3 Generación de la librería

Las librerías de secciones son una herramienta que ofrece MSC.NASTRAN para el uso de manera personal de sus usuarios. MSC.NASTRAN provee de archivos plantilla para que el usuario tenga una referencia de cómo abordar el trabajo, y a partir de los cuales comienza la generación de la librería.

3.1 Ubicación de la plantilla

Antes de comenzar e, se debe buscar la plantilla que proporciona MSC.NASTRAN. Ésta se encuentra en la carpeta **bmsrv**, que a su vez esta dentro del directorio raíz de MSC.NASTRAN. En el caso de ejemplo, la dirección es:

/homesun/locus/cal/msc/old/msc/msc705



Para poder editar los archivos, se copiará el directorio **bmsrv** en el directorio de trabajo. Una vez copiado, se modificaran algunos archivos y se crearan otros nuevos.

3.2 Creación de archivos auxiliares

Se puede identificar cada tipo de archivo .F por las dos primeras letras del nombre. Los archivos que empiezan por "br" son los archivos donde se guardan todas las propiedades de la sección concreta, definida ésta por las dos siguientes letras (en este caso "tu", porque es una sección tube la que incluye el ejemplo MSC.NASTRAN), mientras que la ultima letra indica el tipo de datos que generamos en dicho archivo.

La forma de proceder debe ser ordenada, ya que estarán implicados gran cantidad de archivos, por lo que se recomienda crear una lista con todas las secciones que se vayan a incluir en la librería, y a cada una le asignamos una cadena de dos caracteres (para la sección "tube" se asigna "tu"), siendo letras del abecedario. Se recomienda precaución, ya que no se distingue entre mayúsculas y minúsculas. Aún así tendremos una gran cantidad de combinaciones. A cada sección se le debe asignar también el número de campos que necesitaremos para definir dicha sección (dentro del cual podremos incluir uno que sirva de Switch) y el número de restricciones que se presentan en la geometría (tubo no puede tener radio externo menor que radio interno).

Para cada archivo "br", se crea una copia del "tu" original, y se cambia "tu" por las dos letras asignadas a la sección en cuestión (una copia por sección) y se editará para incluir las propiedades específicas. Esto aplicado al ejemplo será equivalente a crear los siguientes archivos:

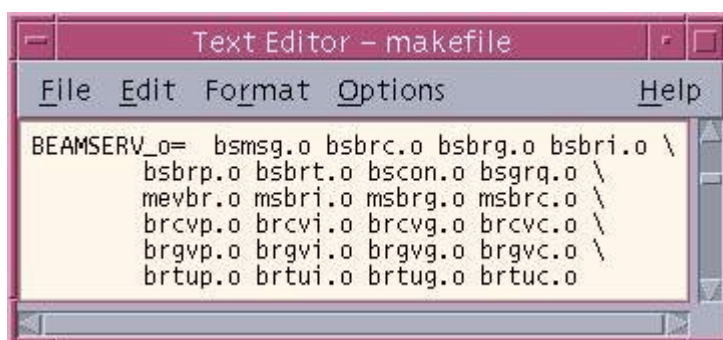
brcvc.F, brcvg.F, brcvi.F, brcvp.F, brgvc.F, brgvg.F, brgvi.F, brgvp.F

3.3 Edición de registro

Se editará el archivo **makefile**, para incluir todos estos archivos en la compilación de la librería. Este archivo es el registro de todos los archivos que el compilador incluirá en el proceso de compilación.



Lo único que interesa del archivo, es ampliar los datos de archivos a compilar, que se guardaran en BEAMSERV_o, quedando este del siguiente modo.



3.4 Descripción de componentes

Después de que todos los archivos necesarios sean incluidos en la compilación, se deberá guiar a la librería para que reconozca las secciones y actúe en consecuencia.

- **bscon.F:** esta rutina genera el número de campos en las líneas de continuación que serán leídos de la tarjeta, tanto para PBARL como PBEAML, para cada sección en la librería. Se aumentará la sentencia IF ya existente en dicho archivo, con los identificadores necesarios, CV y GV (completando una cadena de 8 caracteres con espacios en blanco). Dentro de este IF se indicará el número de dimensiones de la sección con NDIMI (se ha de tener en cuenta, aparte de las dimensiones geométricas, un posible Switch para utilizar la misma sección con distintas salidas; en cuanto a la NSM queda incluido para cualquier sección en una de las últimas líneas de código), en el caso de ejemplo, CV tendrá NDIMI=11 y GV tendrá NDIMI=7.
- **bsbrp.F:** es la interfaz de nuestro calculador de propiedades y MSC.NASTRAN. A partir de este, podemos usar cualquier convención de nombres para las subrutinas de cálculo (en el ejemplo hemos adoptado la convención de MSC.NASTRAN). Este archivo no es necesario modificarlo a menos que se quiera cambiar dicha convención.
- **mevbr.F:** contiene las llamadas a las subrutinas tipo brxxp, escritas en los archivos .F correspondientes. En este archivo se ampliará de nuevo un IF introduciendo otra vez los identificadores de 8 caracteres y a continuación la subrutina que contiene (sustituimos xx por las dos letras de identificación de la sección).
- **mevbm.F:** se creará este archivo como una copia del anterior e introducirá una llamada a este desde bsbrp para el caso de PBEAML, para que el proceso pueda tener lugar. Además, se debe escribir una llamada a la rutina bmxxp, siguiendo un proceso análogo al punto anterior.
- **bsgqr.f:** es una subrutina donde se almacena información para su uso en la maximización de PBARL, esta información es el número de secciones contenidas en la librería y el número máximo de dimensiones que existe para todas ellas. En el caso del ejemplo NSECT=3 y NDIMAX=12 (11+1 en el caso de CV).
- **msbrt.F:** esta rutina suministra el nombre, el número de campos en la tarjeta y el número de restricciones para cada sección en la librería. Esta rutina se usa en la optimización. Se añadirá en este caso CV, 12, 3 para la primera y GV, 8, 5 para la segunda.
- **bsbri.F:** se utiliza si vamos a lanzar una optimización. Provee información para el cálculo de gradientes de las propiedades de la barra en función de sus dimensiones. Los argumentos de salida son dos matrices. La primera, llamada SENTYP que indica: para el valor 0, independencia con respecto a la dimensión correspondiente; para valor 1, variación lineal; para valor 2, variación no lineal; para valor 3 variación desconocida. Para el valor 3, se sabe que la variación es función de dicha dimensión de diseño pero, pero la subrutina bsbrg no provee información analítica del gradiente, con lo que MSC.NASTRAN lo calculará usando diferencias centradas. La segunda matriz, ALIN, es la que proporciona los factores lineales de variación.
- **msbri.F:** contiene las llamadas a las subrutinas tipo brxxi, contenidas en los archivos .F correspondientes. En este archivo se ampliará de nuevo un IF introduciendo otra vez los identificadores y a continuación la subrutina que contiene (sustituimos xx por las dos letras que elegimos en el paso 4º).

- bsbrg.F: esta rutina será necesaria si se lanza una optimización y, para alguna dimensión, si existe variación no lineal conocida de sus propiedades (SENTYP=2), dando los gradientes no lineales de las propiedades de la barra, respecto de sus dimensiones. Dicha información será acumulada en una matriz llamada ANONL.
- bsbrc.F: permite imponer restricciones en valores de las dimensiones de la viga que pueden tomarse como condición de diseño. Sólo es necesaria si se utiliza optimización e, incluso entonces, es posible solo imponer condiciones lineales (sin termino independiente, ejemplo: $a \cdot \text{DIM1} + b \cdot \text{DIM2} \leq 0$, donde a y b son los coeficientes que obtendremos) entre las dimensiones de la sección, y siempre en un número igual o menor al guardado en la subrutina bsbrt.
- msbrc.F: contiene las llamadas a las subrutinas tipo brxxc, contenidas, a su vez, en los archivos .F correspondientes. En este archivo se ampliará de nuevo un IF introduciendo otra vez los identificadores y a continuación la subrutina que contiene (sustituimos xx por las dos letras que elegimos en el paso 4º).
- bsmsg.F: es el gestor de los errores. Cuando se produce un error en cualquiera de las rutinas anteriores, este llega aquí, es interpretado y, acto seguido, produce un mensaje de error que se escribirá en el archivo .f06 correspondiente.

3.5 Edición de propiedades

Este apartado incluye el código fuente de la sección CV, que servirá como guía de la edición. Este código se escribirá en el archivo correspondiente a la sección, para este ejemplo, en los archivos: brcvc.F, brcvg.F, brcvi.F, brcvp.F

El modo de proceder será, primero presentando el código original de la plantilla, y luego el código modificado. Se suprimirán todas las líneas que vayan precedidas de C, ya que este símbolo denota comentario.

- brcvp.F:

```
#ifndef _DOUBLE
    SUBROUTINE BRTUPD(ID,NID,IDO,NIDO,DIMI,NDIMI,DIMO,NDIMO,ERROR)
#else
    SUBROUTINE BRTUPS(ID,NID,IDO,NIDO,DIMI,NDIMI,DIMO,NDIMO,ERROR)
#endif
    IMPLICIT INTEGER (I-N)
#ifdef _DOUBLE
    IMPLICIT DOUBLE PRECISION (A-H,O-Z)
#else
    IMPLICIT REAL (A-H,O-Z)
#endif
    INTEGER ERROR,ID(NID),IDO(NIDO)
    INTEGER NAM(2)
#ifdef _DOUBLE
    DOUBLE PRECISION CO,DS,CT
#else
    REAL CO,DS,CT
#endif
#ifdef _DOUBLE
    DOUBLE PRECISION DIMI(NDIMI),DIMO(NDIMO)
#else
    REAL DIMI(NDIMI),DIMO(NDIMO)
#endif
#ifdef _DOUBLE
    COMMON /CONDAD/PI
#else
    COMMON /CONDAS/PI
#endif
#ifdef _DOUBLE
    DATA NAM/4HBRTU,4HPD /
    CO = 0.D0
    DS = 2.D0
#else
    DATA NAM/4HBRTU,4HPS /
    CO = 0.E0
    DS = 2.E0
#endif
    DO 30 II = 1,NID
        IDO(II) = ID(II)
    30 CONTINUE
    IF ( DIMI(1).LE.DIMI(2) ) GO TO 100
    IF ( DIMI(2).LT.CO ) GO TO 99
    CT = DS*DS
    DIM1 = DIMI(1)
    DIM2 = DIMI(2)
    DIMO(1) = PI*(DIM1*DIM1-DIM2*DIM2)
    DIMO(2) = PI*(DIM1**CT-DIM2**CT)/CT
```



```

        DIMO(4) = DIMO(2)*DS
        DIMO(15) = DS/CT
        DIMO(16) = DS/CT
        DIMO(6) = CO
        DIMO(8) = CO
        DIMO(9) = CO
        DIMO(12) = CO
        DIMO(13) = CO
        DIMO(17) = CO
        DIMO(3) = DIMO(2)
        DIMO(5) = DIMI(3)
        DIMO(7) = DIM1
        DIMO(10) = DIM1
        DIMO(11) = -DIM1
        DIMO(14) = -DIM1
        GO TO 200
99      ERROR = 5101
        GO TO 200
100     ERROR = 5102
200     RETURN
        END

```

Puede extrañar, al conocedor del lenguaje FORTRAN, algunas de las peculiaridades de este código, como pueden ser la letra C antes de la línea comentada o el carácter #. Para aclarar esto, hay que decir que MSC.NASTRAN no compila directamente los archivos que le suministramos, primero utiliza un script, el cual lee el documento .F y lo escribe en otro documento para posteriormente compilarlo. Hace esto para no tener que utilizar distintos archivos para distintas funciones, dependiendo de la precisión que se le requieran a estas. La utilidad de # es poder ser interpretado por el script como separador de la parte de código en la cual utilizamos variables REAL, las cuales pueden estar en doble o simple precisión. Una manera de minimizar este código duplicado es duplicar solamente la declaración de variables, declarar todos los números que vayamos a usar y utilizar en todas las operaciones posteriores variables declaradas. Esta forma de proceder queda reflejada en el código adjunto, el cual difiere del original mínimamente.

```

#ifdef _DOUBLE
        SUBROUTINE BRCVPD(ID,NID,IDO,NIDO,DIMI,NDIMI,DIMO,NDIMO,ERROR)
#else
        SUBROUTINE BRCVPS(ID,NID,IDO,NIDO,DIMI,NDIMI,DIMO,NDIMO,ERROR)
#endif
        IMPLICIT INTEGER (I-N)
#ifdef _DOUBLE
        IMPLICIT DOUBLE PRECISION (A-H,O-Z)
#else
        IMPLICIT REAL (A-H,O-Z)
#endif
        INTEGER ERROR,ID(NID),IDO(NIDO)
        INTEGER NAM(2)
#ifdef _DOUBLE
        DOUBLE PRECISION DIM1,DIM2,DIM3,DIM4,DIM5,Xg,Yg,ac,CO,MD,DS,TS,DC,
+                               Switch
#else
        REAL DIM1,DIM2,DIM3,DIM4,DIM5,Xg,Yg,ac,CO,MD,DS,TS,DC,
+                               Switch
#endif
#ifdef _DOUBLE
        DOUBLE PRECISION DIMI(NDIMI),DIMO(NDIMO),Ar(5),Xi(5),Yi(5),th(5),
+                               Lx(5),Ly(5)
#else
        REAL DIMI(NDIMI),DIMO(NDIMO),Ar(5),Xi(5),Yi(5),th(5),
+                               Lx(5),Ly(5)

```

```
#endif
#ifdef _DOUBLE
COMMON /CONDAD/PI
#else
COMMON /CONDAS/PI
#endif
#ifdef _DOUBLE
DATA NAM/4HBRCV,4HPD /
CO = 0.D0
MD = 0.5D0
DS = 2.D0
TS = 3.D0
DC = 12.D0
#else
DATA NAM/4HBRCV,4HPS /
CO = 0.E0
MD = 0.5E0
DS = 2.E0
TS = 3.E0
DC = 12.E0
#endif
DO 30 II = 1,NID
ID0(II) = ID(II)
30 CONTINUE
IF ( DIMI(1).LT.CO ) GO TO 100
IF ( DIMI(5).LT.CO ) GO TO 100
IF ( DIMI(6).LT.CO ) GO TO 100
IF ( DIMI(7).LT.CO ) GO TO 100
IF ( DIMI(8).LT.CO ) GO TO 100
IF ( DIMI(9).LT.CO ) GO TO 100
IF ( DIMI(10).LT.CO ) GO TO 100
IF ( DIMI(3).LT.(DIMI(2)+DIMI(4)) ) GO TO 101
IF ( DIMI(2).LT.DIMI(6) ) GO TO 102
IF ( DIMI(4).LT.DIMI(10) ) GO TO 103
DIM1 = DIMI(1)
DIM2 = DIMI(2)
DIM3 = DIMI(3)
DIM4 = DIMI(4)
DIM5 = DIMI(5)
th(1) = DIMI(6)
th(2) = DIMI(7)
th(3) = DIMI(8)
th(4) = DIMI(9)
th(5) = DIMI(10)
Switch = DIMI(11)
Lx(1) = DIM1
Ly(1) = th(1)
Lx(2) = th(2)
Ly(2) = DIM2-th(1)
Lx(3) = th(3)
Ly(3) = DIM3-DIM2-DIM4
Lx(4) = th(4)
Ly(4) = DIM4-th(5)
Lx(5) = DIM5
Ly(5) = th(5)
Xg = CO
Yg = CO
ac = CO
DO 40 II=1,5
Ar(II) = Lx(II)*Ly(II)
Xi(II) = CO
Yi(II) = CO
DO 46 IJ=1,II-1
```

```

        Yi(II) = Yi(II)+Ly(IJ)
46      CONTINUE
        Xi(II) = Lx(II)/DS
        Yi(II) = Yi(II)+Ly(II)/DS
        Xg = Xg+Ar(II)*Xi(II)
        Yg = Yg+Ar(II)*Yi(II)
        ac = ac+Ar(II)
40      CONTINUE
        DIMO(1) = ac
        Xg = Xg/ac
        Yg = Yg/ac
        ac = C0
        DO 45 II = 1,5
            Xi(II) = Xi(II)-Xg
            Yi(II) = Yi(II)-Yg
45      CONTINUE
        DO 50 II = 1,5
            ac = ac+Ar(II)*Yi(II)**DS+Ar(II)*Ly(II)**DS/DC
50      CONTINUE
        DIMO(2) = ac
        IF (Switch.GT.C0) THEN
            GO TO 200
        END IF
        ac = C0
        DO 60 II = 1,5
            ac = ac+Ar(II)*Xi(II)**DS+Ar(II)*Lx(II)**DS/DC
60      CONTINUE
        DIMO(3) = ac
        ac = C0
        DO 70 II = 1,5
            ac = ac+Ar(II)*th(II)**DS/TS
70      CONTINUE
        DIMO(4) = ac
        DIMO(5) = DIMI(11)
        ac = C0
        DO 80 II = 1,5
            ac = ac+Ar(II)*Xi(II)*Yi(II)
80      CONTINUE
        DIMO(17) = ac
        DIMO(8) = DIM5-Xg
        DIMO(7) = DIM3-Yg
        DIMO(10) = DIM1-Xg
        DIMO(9) = -Yg
        DIMO(12) = -Xg
        DIMO(11) = -Yg
        DIMO(14) = -Xg
        DIMO(13) = DIM3-Yg
        DIMO(15) = (Lx(2)*Ly(2)+Lx(3)*Ly(3)+Lx(4)*Ly(4))/DIMO(1)
        DIMO(16) = ((Lx(1)-th(1))*Ly(1)+(Lx(5)-th(5))*Ly(5))/DIMO(1)
        GO TO 200
100     ERROR = 5103
        GO TO 200
101     ERROR = 5104
        GO TO 200
102     ERROR = 5105
        GO TO 200
103     ERROR = 5106
200     RETURN
END

```

Para el código de la nueva sección, lo primero es cambiar el nombre de las subrutinas, sustituyendo la cadena "TU" por la cadena correspondiente, "CV" en este caso. A continuación, se declaran las

variables necesarias para el cálculo de propiedades. La declaración no es única, sino que puede variar de un programador a otro, la que se presenta en el ejemplo es una sugerencia y depende enormemente del resto del código. En este caso, la forma de operar consiste en dividir la sección en rectángulos, evaluar las propiedades de cada uno y sumarlas luego de manera apropiada. Por lo que se entiende que de DIM1 a DIM5 son las longitudes de la sección y el vector t es un vector de espesores (se han utilizado dos posibles métodos de declaración aunque se recomienda el uso del segundo), X_g e Y_g son las coordenadas del centro de gravedad de la sección, ac es un simple acumulador para realizar sumas; CO, MD, DS, TS y DC son constantes iguales al número real 0, 0.5, 2, 3 y 12; Switch será el interruptor para las distintas modalidades de cálculo; Ar, Xi, Yi, Lx y Ly son vectores donde se guardaran las propiedades de los cinco rectángulos elementales (área, coordenada según x, coordenada según y, longitud según x, longitud según y).

El sistema de referencia tiene su origen en el vértice inferior izquierdo de la sección y está orientado según ejes paralelos a los bordes inferior e izquierdo de la sección.

Tras la declaración y una operación interna heredada del proceso plantilla, lo primero es comprobar posibles errores, como pueden ser: dimensiones negativas o incumplimiento de restricciones geométricas. El siguiente paso es calcular las propiedades de las que se habla en la declaración de variables. Se ha de prestar especial atención a la obtención del área y coordenadas del centro de gravedad de la sección, para las cuales se utiliza un bucle FOR para sumar acumulativamente.

Hay que tener presente que no se puede omitir la asignación de datos a las variables de salida, puede se hayan calculado todas las propiedades pero se olvide dar dichos valores a las variables DIMO, lo que llevaría a que la tarjeta PBAR correspondiente estuviera vacía. Tras calcular el área, como suma de las áreas elementales y la inercia I_{ZZBAR} como suma de inercias elementales más la contribución a la separación del centro de gravedad, entra en funcionamiento el Switch, el cual para un valor superior a cero dirige el flujo de datos al final de la rutina, saltándose el resto de cálculos. Los siguientes pasos son el cálculo del resto de propiedades: I_{YYBAR} , J, I_{ZYBAR} , coordenadas de puntos extremos (C, D, E, F), K_1 y K_2 .

Las últimas líneas derivan los errores hacia la rutina bsmg.F, de donde se sacaran el texto correspondiente a cada error.

- brcvi.F:

```
#ifdef _DOUBLE
    SUBROUTINE BRTUID(SENTYP,ALIN,NDIM,NPROP,ERROR)
#else
    SUBROUTINE BRTUIS(SENTYP,ALIN,NDIM,NPROP,ERROR)
#endif
    IMPLICIT INTEGER (I-N)
#ifdef _DOUBLE
    IMPLICIT DOUBLE PRECISION (A-H,O-Z)
#else
    IMPLICIT REAL (A-H,O-Z)
#endif
    INTEGER ERROR,SENTYP(NPROP,NDIM)
#ifdef _DOUBLE
    DOUBLE PRECISION ALIN(NPROP,NDIM),UN
#else
    REAL ALIN(NPROP,NDIM),UN
#endif
    INTEGER NAM(2)
#ifdef _DOUBLE
    DATA NAM/4HBRTU,4HID /
```

```

        UN = 1.0D0
#else
        DATA NAM/4HBRTU,4HIS /
        UN = 1.0E0
#endif
        ALIN( 7,1) = UN
        ALIN(10,1) = UN
        ALIN(11,1) = -UN
        ALIN(14,1) = -UN
        ALIN( 5,3) = UN
        SENTYP( 1,1) = 2
        SENTYP( 1,2) = 2
        SENTYP( 2,1) = 2
        SENTYP( 2,2) = 2
        SENTYP( 3,1) = 2
        SENTYP( 3,2) = 2
        SENTYP( 4,1) = 2
        SENTYP( 4,2) = 2
        SENTYP( 7,1) = 1
        SENTYP(10,1) = 1
        SENTYP(11,1) = 1
        SENTYP(14,1) = 1
        SENTYP( 5,3) = 1
        RETURN
        END

```

Teniendo siempre en mente el archivo anterior, el cual muestra todas las relaciones entre las propiedades y las dimensiones, se deben establecer cuales de estas tienen una relación lineal, constante, no lineal conocida ó no conocida (esto obligará a MSC.NASTRAN a calcular gradientes con deferencias centradas). En el caso de CV todas las propiedades, excepto NSM, tienen relaciones no lineales por lo que para todas sus propiedades deberán tener SENTYP igual a 2, aunque si no se incluye un archivo donde se exprese analíticamente, se puede asignar el valor 3 (en este ejemplo, por mero interés didáctico, se desarrollará el gradiente de modo analítico pero se recomienda, siempre que la relación sea no lineal, asignar el valor 3).

```

#ifdef _DOUBLE
        SUBROUTINE BRCVID(SENTYP,ALIN,NDIM,NPROP,ERROR)
#else
        SUBROUTINE BRCVIS(SENTYP,ALIN,NDIM,NPROP,ERROR)
#endif
        IMPLICIT INTEGER (I-N)
#ifdef _DOUBLE
        IMPLICIT DOUBLE PRECISION (A-H,O-Z)
#else
        IMPLICIT REAL (A-H,O-Z)
#endif
        INTEGER ERROR,SENTYP(NPROP,NDIM)
#ifdef _DOUBLE
        DOUBLE PRECISION ALIN(NPROP,NDIM),UN
#else
        REAL ALIN(NPROP,NDIM),UN
#endif
        INTEGER NAM(2)
#ifdef _DOUBLE
        DATA NAM/4HBRTU,4HID /
        UN = 1.0D0
#else
        DATA NAM/4HBRTU,4HIS /
        UN = 1.0E0
#endif

```

```

        ALIN( 5,12) = UN
        SENTYP( 5,12) = 1
        DO 40 II=1,4
            DO 46 IJ=1,10
                SENTYP( II,IJ) = 2
46        CONTINUE
40        CONTINUE
        DO 50 II=7,16
            DO 56 IJ=1,10
                SENTYP( II,IJ) = 2
56        CONTINUE
50        CONTINUE
        RETURN
        END

```

Como para este caso todas las propiedades, excepto NSM y FE, tienen un valor 2, se ha utilizado dos bucles para asignarlo de forma más compacta.

- brcvg.F:

```

#ifdef _DOUBLE
    SUBROUTINE BRTUGD(DIMI,NDIMI,ANONL,NPROP,ERROR)
#else
    SUBROUTINE BRTUGS(DIMI,NDIMI,ANONL,NPROP,ERROR)
#endif
    IMPLICIT INTEGER (I-N)
#ifdef _DOUBLE
    IMPLICIT DOUBLE PRECISION (A-H,O-Z)
#else
    IMPLICIT REAL (A-H,O-Z)
#endif
    INTEGER NDIMI,NPROP,ERROR
#ifdef _DOUBLE
    DOUBLE PRECISION ANONL(NPROP,NDIMI),DIMI(NDIMI),DS,TS
#else
    REAL ANONL(NPROP,NDIMI),DIMI(NDIMI),DS,TS
#endif
    INTEGER NAM(2)
#ifdef _DOUBLE
    COMMON /CONDAD/ PI
    DS = 2.0D0
    TS = 3.0D0
#else
    COMMON /CONDAS/ PI
    DS = 2.0E0
    TS = 3.0E0
#endif
#ifdef _DOUBLE
    DATA NAM/4HBRTU,4HGD /
#else
    DATA NAM/4HBRTU,4HGS /
#endif
    DIM1 = DIMI(1)
    DIM2 = DIMI(2)
    PDIM13 = PI*DIM1**TS
    PDIM23 = PI*DIM2**TS
    ANONL(1,1) = DS*PI*DIM1
    ANONL(1,2) = -DS*PI*DIM2
    ANONL(4,1) = DS*PDIM13
    ANONL(4,2) = -DS*PDIM23
    ANONL(2,1) = PDIM13

```

```

ANONL(2,2) = -PDIM23
ANONL(3,1) = PDIM13
ANONL(3,2) = -PDIM23
RETURN
END

```

El primer paso, como en todos los archivos anteriores, debe ser sustituir, en la cabecera, los nombres de las rutinas ("TU" por "CV"). También se recomienda declarar las variables que se vayan a usar, aunque se hayan declarado implícitamente al principio de la rutina.

En cuanto al cálculo del gradiente, se vuelve a recomendar el uso numérico de MSC.NASTRAN, pero si por el contrario este se quiere expresar de forma analítica, ha de ponerse especial atención al primero de los archivos modificados, a partir del cual se pueden aprovechar muchas fórmulas.

```

#ifdef _DOUBLE
    SUBROUTINE BRCVGD(DIMI,NDIMI,ANONL,NPROP,ERROR)
#else
    SUBROUTINE BRCVGS(DIMI,NDIMI,ANONL,NPROP,ERROR)
#endif
    IMPLICIT INTEGER (I-N)
#ifdef _DOUBLE
    IMPLICIT DOUBLE PRECISION (A-H,O-Z)
#else
    IMPLICIT REAL
        (A-H,O-Z)
#endif
    INTEGER
        NDIMI,NPROP,ERROR
#ifdef _DOUBLE
    DOUBLE PRECISION ANONL(NPROP,NDIMI),DIMI(NDIMI)
#else
    REAL
        ANONL(NPROP,NDIMI),DIMI(NDIMI)
#endif
    INTEGER NAM(2)
#ifdef _DOUBLE
    DOUBLE PRECISION DIM1,DIM2,DIM3,DIM4,DIM5,Xg,Yg,ac,CO,MD,DS,TS,DC,
+
        Switch,UN,AT
#else
    REAL
        DIM1,DIM2,DIM3,DIM4,DIM5,Xg,Yg,ac,CO,MD,DS,TS,DC,
+
        Switch,UN,AT
#endif
#ifdef _DOUBLE
    DOUBLE PRECISION Ar(5),Xi(5),Yi(5),Ldy(5,10),Xdg(10),Ydg(10),
+
        Lx(5),Ly(5),Xdi(5,10),Ydi(5,10),Adr(5,10),th(5)
#else
    REAL
        Ar(5),Xi(5),Yi(5),Ldy(5,10),Xdg(10),Ydg(10),
+
        Lx(5),Ly(5),Xdi(5,10),Ydi(5,10),Adr(5,10),th(5)
#endif
#ifdef _DOUBLE
    COMMON /CONDAD/ PI
#else
    COMMON /CONDAS/ PI
#endif
#ifdef _DOUBLE
    DATA NAM/4HBRCV,4HPD /
    CO = 0.D0
    MD = 0.5D0
    UN = 1.0D0
    DS = 2.D0
    TS = 3.D0
    DC = 12.D0
#else
    DATA NAM/4HBRCV,4HPS /

```

```
CO = 0.E0
MD = 0.5E0
UN = 1.0E0
DS = 2.E0
TS = 3.E0
DC = 12.E0
#endif
DIM1 = DIMI(1)
DIM2 = DIMI(2)
DIM3 = DIMI(3)
DIM4 = DIMI(4)
DIM5 = DIMI(5)
th(1) = DIMI(6)
th(2) = DIMI(7)
th(3) = DIMI(8)
th(4) = DIMI(9)
th(5) = DIMI(10)
Switch = DIMI(11)
Lx(1) = DIM1
Ly(1) = th(1)
Lx(2) = th(2)
Ly(2) = DIM2-th(1)
Lx(3) = th(3)
Ly(3) = DIM3-DIM2-DIM4
Lx(4) = th(4)
Ly(4) = DIM4-th(5)
Lx(5) = DIM5
Ly(5) = th(5)
Xg = CO
Yg = CO
Adr(1,1) = th(1)
Adr(1,6) = Lx(1)
Adr(2,2) = th(2)
Adr(2,6) = -th(2)
Adr(2,7) = Ly(2)
Adr(3,2) = -th(3)
Adr(3,3) = th(3)
Adr(3,4) = -th(3)
Adr(3,8) = Ly(3)
Adr(4,4) = th(4)
Adr(4,9) = Ly(4)
Adr(4,10) = -th(4)
Adr(5,5) = th(5)
Adr(5,10) = Lx(5)
Xdi(1,1) = MD
Xdi(2,7) = MD
Xdi(3,8) = MD
Xdi(4,9) = MD
Xdi(5,5) = MD
Ydi(1,6) = MD
Ydi(2,2) = MD
Ydi(2,6) = MD
Ydi(3,2) = MD
Ydi(3,3) = MD
Ydi(3,4) = -MD
Ydi(4,3) = UN
Ydi(4,4) = MD
Ydi(4,10) = -MD
Ydi(5,3) = UN
Ydi(5,10) = -MD
Ldy(1,6) = UN
Ldy(2,2) = UN
Ldy(2,6) = -UN
```



```

Ldy(3,2) = -UN
Ldy(3,3) = UN
Ldy(3,4) = -UN
Ldy(4,4) = UN
Ldy(4,10) = -UN
Ldy(5,10) = UN
DO 51 II=1,10
    ac = C0
    DO 56 IJ=1,5
        ac = ac+Adr(IJ,II)
56    CONTINUE
    ANONL(1,II) = ac
51    CONTINUE
    ac=C0
    DO 41 II=1,5
        Ar(II) = Lx(II)*Ly(II)
        Xi(II) = C0
        Yi(II) = C0
        DO 46 IJ=1,II-1
            Yi(II) = Yi(II)+Ly(IJ)
46    CONTINUE
        Xi(II) = Lx(II)/DS
        Yi(II) = Yi(II)+Ly(II)/DS
        Xg = Xg+Ar(II)*Xi(II)
        Yg = Yg+Ar(II)*Yi(II)
        ac = ac+Ar(II)
41    CONTINUE
    AT = ac
    Xg = Xg/ac
    Yg = Yg/ac
    ac = C0
    DO 45 II = 1,5
        Xi(II) = Xi(II)-Xg
        Yi(II) = Yi(II)-Yg
45    CONTINUE
    DO 52 II=1,10
        ac = C0
        DO 57 IJ=1,5
            ac = ac+Adr(IJ,II)*(Yi(IJ)**DS+Ly(IJ)**DS/DC)
            ac = ac+DS*Ar(IJ)*(Yi(IJ)*Ydi(IJ,II)+Ly(IJ)*Ldy(IJ,II)/DC)
57    CONTINUE
        ANONL(2,II) = ac
52    CONTINUE
    ac = C0
    DO 53 II=1,10
        ac = C0
        DO 58 IJ=1,5
            ac = ac+Adr(IJ,II)*(Xi(IJ)**DS+Lx(IJ)**DS/DC)
            ac = ac+DS*Ar(IJ)*(Xi(IJ)+Ly(IJ)*DS/DC)*Xdi(IJ,II)
58    CONTINUE
        ANONL(3,II) = ac
53    CONTINUE
    ac=C0
    DO 54 II=1,10
        ac = C0
        DO 59 IJ=1,5
            ac = ac+Adr(IJ,II)*th(IJ)**DS/TS
59    CONTINUE
    IF (II.GT.5) THEN
        ANONL(4,II) = ac+DS*Ar(II-5)*th(II-5)
    ELSE
        ANONL(4,II) = ac
    END IF

```

```

54  CONTINUE
    ac=C0
    DO 42 II=1,10
        ac = C0
        DO 47 IJ=1,5
            Xdg(II) = Xdg(II)+Xi(IJ)*Adr(IJ,II)+Ar(IJ)*Xdi(IJ,II)/AT
            Ydg(II) = Ydg(II)+Yi(IJ)*Adr(IJ,II)+Ar(IJ)*Ydi(IJ,II)/AT
47  CONTINUE
        DO 43 IK=7,13,2
            ANONL(IK+1,II) = -Xdg(II)
            ANONL(IK,II) = -Ydg(II)
43  CONTINUE
42  CONTINUE
    ANONL(8,5)=ANONL(8,5)+1
    ANONL(7,3)=ANONL(7,3)+1
    ANONL(10,1)=ANONL(10,1)+1
    ANONL(13,3)=ANONL(13,3)+1
    DO 44 II=1,10
        ANONL(15,II) = Adr(2,II)+Adr(2,II)+Adr(2,II)
        ac = (Ar(2)+Ar(2)+Ar(2))*ANONL(1,II)/AT
        ANONL(15,II) = ANONL(15,II)-ac
        ANONL(15,II) = ANONL(15,II)/AT
        ac = ((Lx(1)-th(1))*Ly(1)+(Lx(5)-th(5))*Ly(5))/AT
        ANONL(16,II) = -ac
        ANONL(16,II) = ANONL(16,II)*ANONL(1,II)/AT
        ac = (Adr(1,II)+Adr(5,II)-th(1)*Ldy(1,II)-th(5)*Ldy(5,II))
        ANONL(16,II) = ANONL(16,II)+ac/AT**DS
44  CONTINUE
    ANONL(16,6) = ANONL(16,6)-(Ly(1)+Ly(5))/AT**DS
    ac=C0
200  RETURN
    END

```

Como se puede apreciar este código es largo y complicado de depurar, por lo que se vuelve a hacer hincapié en el cálculo numérico de las derivadas, el cual da una precisión suficiente.

Todo éste código se utiliza para el cálculo de las derivadas de distintas variables que intervienen en las propiedades.

Una vez calculadas todas las variables auxiliares, se asignan los valores a la variable ANONL, donde acumulamos el jacobiano respecto de las propiedades geométricas de la sección.

- brcvc.F:

```

#if defined _DOUBLE
    SUBROUTINE BRTUCD(AFACT,NCONST,NDIMI,ERROR)
#else
    SUBROUTINE BRTUCS(AFACT,NCONST,NDIMI,ERROR)
#endif
    IMPLICIT INTEGER (I-N)
#ifdef _DOUBLE
    IMPLICIT DOUBLE PRECISION (A-H,O-Z)
#else
    IMPLICIT REAL (A-H,O-Z)
#endif
    INTEGER NCONST,NDIMI,ERROR,UN
#ifdef _DOUBLE
    DOUBLE PRECISION AFACT(NCONST,NDIMI)
#else
    REAL AFACT(NCONST,NDIMI)

```

```
#endif
      INTEGER NAM(2)
#ifdef _DOUBLE
      DATA NAM/4HBRTU,4HCD  /
      UN = 1.0D0
#else
      DATA NAM/4HBRTU,4HCS  /
      UN = 1.0E0
#endif
      IF(NCONST.NE.1) THEN
        ERROR = 5502
        RETURN
      END IF
      AFACT(1,1) = -UN
      AFACT(1,2) =  UN
      RETURN
      END
```

Comenzaremos por lo que debe ser ya un proceso mecánico, sustituir la cadena "TU" por la correspondiente de la sección particular, en este caso "CV". Ya que este es el último archivo que modificaremos, referente a propiedades de secciones, solamente recordar que también hay que cambiar el nombre de los archivos, no solamente el encabezado del contenido.

```
#if defined _DOUBLE
      SUBROUTINE BRCVCD(AFACT,NCONST,NDIMI,ERROR)
#else
      SUBROUTINE BRCVCS(AFACT,NCONST,NDIMI,ERROR)
#endif
      IMPLICIT INTEGER (I-N)
#ifdef _DOUBLE
      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
#else
      IMPLICIT REAL          (A-H,O-Z)
#endif
      INTEGER                NCONST,NDIMI,ERROR,UN
#ifdef _DOUBLE
      DOUBLE PRECISION      AFACT(NCONST,NDIMI)
#else
      REAL                   AFACT(NCONST,NDIMI)
#endif
      INTEGER NAM(2)
#ifdef _DOUBLE
      DATA NAM/4HBRTU,4HCD  /
      UN = 1.0D0
#else
      DATA NAM/4HBRTU,4HCS  /
      UN = 1.0E0
#endif
      IF(NCONST.NE.3) THEN
        ERROR = 5502
        RETURN
      END IF
      AFACT(1,3) = -UN
      AFACT(1,2) =  UN
      AFACT(1,4) =  UN
      AFACT(2,2) = -UN
      AFACT(2,6) =  UN
      AFACT(3,4) = -UN
      AFACT(3,10) =  UN
      RETURN
      END
```

En éste, el último archivo de cada sección, solo tenemos que introducir los coeficientes lineales y no nulos de la relación matricial que se deriva de las restricciones geométricas de la sección en estudio. En esta ocasión, la matriz será AFACT.

3.6 Compilación

Una vez que se disponga de todos los archivos necesarios, será posible "linkar" la librería, mejor dicho, construiremos un beamserv. Con una consola, nos ubicaremos en la carpeta donde estén almacenados todos los archivos (copia de la plantilla y ya editados), y se ejecutará el siguiente comando:

msc705 bmsrv build

Esta orden comenzará a generar archivos .o con el mismo nombre que los .F, necesarios para la compilación. Al final, quedará un archivo ejecutable con nombre beamserv, necesario en la evaluación de la tarjeta PBARL que generemos.

4 Instalación

Este tipo de aplicaciones es muy flexible, por lo que no será necesario un seguimiento estricto de este epígrafe para que se logre ejecutar el beamserv. Lo que se expone a continuación es un proceso por el cual todos los potenciales usuarios pueden tener acceso a la librería.

4.1 Ubicación e instalación de la librería

El lugar idóneo de ubicación sería el propio directorio de instalación de MSC.NASTRAN. Simplemente se almacenaría el archivo beamserv en una carpeta, a la cual denominaremos con un criterio que permita distinguirla de otras librerías. Para la ejecución de una base de datos bajo MSC.NASTRAN que requiera la librería, solo habría que facilitar un archivo como el descrito en el apartado correspondiente al uso de las librerías. Si este archivo no estuviera en el poder del usuario el mismo podría crearlo sin mas necesidad que el conocimiento del path correcto. Una recomendación es ubicar un archivo, el cual pueda copiar todo usuario y que acompañe a su base de datos.

5 Validación de resultados

5.1 Ejemplo 1: equivalencia sección C Variable con CHAN

Se adjunta el caso de un modelo muy sencillo (viga en ménsula) que utiliza una sección CV con la peculiaridad de tener todos los espesores y ambas alas iguales, lo que la hace equivalente a un CHAN.

	DIM1	DIM2	DIM3	DIM4	DIM5	DIM6	DIM7	DIM8	DIM9	DIM10	Switch	NSM
CV	0.1	0.05	0.1	0.05	0.1	0.01	0.01	0.01	0.01	0.01	0.	

Archivo .pch:

```
PBAR      *          1          1  2.79999990E-03  4.49333356E-06
*          2.83047621E-06  9.33333268E-08  0.00000000E+00
*          5.00000007E-02  6.28571436E-02 -5.00000007E-02  6.28571436E-02
*          -5.00000007E-02 -3.71428579E-02  5.00000007E-02 -3.71428579E-02
*          2.85714298E-01  6.42857134E-01  1.05879118E-22
*
```

	DIM1	DIM2	DIM3	DIM4	NSM
CHAN	0.1	0.1	0.01	0.01	

Archivo .pch:

```
PBAR      *          1          1  2.79999990E-03  4.49333356E-06
*          2.83047621E-06  9.33333268E-08  0.00000000E+00
*          5.00000007E-02  6.28571436E-02 -5.00000007E-02  6.28571436E-02
*          -5.00000007E-02 -3.71428579E-02  5.00000007E-02 -3.71428579E-02
*          2.85714298E-01  6.42857134E-01  0.00000000E+00
*
```

Se puede apreciar que hay una concordancia total en todos los campos, excepto en el campo relacionado con el producto de inercia, el cual es despreciable frente al resto de inercias, pero no es cero debido a que en el proceso de cálculo se generan errores del orden de la precisión de la máquina (se está operando con órdenes de las propiedades y el área, lo que equivale a 1.0E-4), mientras que MSC.NASTRAN se le asigna el valor cero sin necesidad de cálculo.

5.2 Ejemplo 2: sección C Variable calculada con AIDA

Para el mismo modelo, cambiamos las propiedades de la sección. El resultado se compara con los valores obtenidos del programa AIDA.

	DIM1	DIM2	DIM3	DIM4	DIM5	DIM6	DIM7	DIM8	DIM9	DIM10	Switch	NSM
CV	0.1	0.07	0.2	0.04	0.12	0.007	0.01	0.006	0.011	0.0125	0.	

Archivo .pch:

```
PBAR      *              1              1  3.67249991E-03  2.33248938E-05
*          4.86917997E-06  1.29239169E-07  0.00000000E+00
*          8.23718533E-02  8.42113644E-02 -1.17628150E-01  6.42113686E-02
*          -1.17628150E-01 -3.57886329E-02  8.23718533E-02 -3.57886329E-02
*          4.00953025E-01  5.43158591E-01  2.69634256E-06
*
```

Resultados AIDA:

A	0.0036725	Iyy	0.000023325	Izz	0.000004869
Iyz	0.000002696	Ycg	0.03579	Zcg	0.11763
I ₁₁	0.000023325	I ₂₂	0.000004869	Ángulo	-8.144

Comparando resultados, la precisión es del orden del software AIDA.

5.3 Ejemplo 3: sección G Variable calculada con AIDA

Para el mismo modelo, cambiamos la sección, con lo que deberemos reducir el número de dimensiones. El resultado se compara con los valores obtenidos del programa AIDA.

	DIM1	DIM2	DIM3	DIM4	DIM5	DIM6	Switch	NSM
CV	0.5	1	2	0.7	0.1	0.1	0.	

Archivo .pch:

```
PBAR      *          1          1  3.70685846E-01  1.96928829E-01
*          3.96743268E-02  1.23561954E-03  0.00000000E+00
*          1.13537848E+00  3.89923394E-01 -8.64621520E-01  6.89923406E-01
*          -8.64621520E-01 -3.10076594E-01  1.13537848E+00 -3.10076594E-01
*          5.12563407E-01  2.96747237E-01 -2.90899388E-02
*
```

Resultados AIDA:

A	0.37069	Iyy	0.19693	Izz	0.03967
Iyz	-0.02909	Ycg	0.3101	Zcg	0.8646
I ₁₁	0.20214	I ₂₂	0.03447	Ángulo	10.152

Comparando resultados, la precisión es del orden del software AIDA.